Sprint 2 Requirements Artifacts

Instacart Grocery Delivery Game

Sprint 2 Goal

Implement the shopping system: grocery inventory, aisle item selection, randomized orders, checklist, cart, order validation, countdown timer, and earnings.

Sprint 2 Requirements (from Initial Requirements spreadsheet)

Requirement 7 — Create database of grocery aisles, items, prices, and stock quantities — 8 points
Requirement 8 — Display shelf items for the selected aisle and allow item selection — 5 points
Requirement 9 — Generate customer orders with randomized grocery lists and quantities — 8 points
Requirement 10 — Show active order checklist and update collected-item progress — 5 points
Requirement 11 — Allow players to choose item quantities and add or remove items from cart — 5 points
Requirement 12 — Validate cart against order requirements and identify missing or incorrect items — 5 points
Requirement 16 — Add visible countdown timer and order expiration rules — 5 points
Requirement 17 — Calculate order earnings and tips based on speed and order accuracy — 8 points
Total: 52 story points
The requirement IDs, descriptions, story points, and sprint assignments above match the supplied spreadsheet. Functional and technical details below are proposed implementations.

Artifact 1 — Requirement 7: Grocery Aisle and Item Database

A. Requirement Information

Requirement ID: R7
Requirement Name: Grocery Aisle and Item Database
Description: Create a database containing grocery aisles, items, prices, and stock quantities.
Story Points: 8

B. Functional Details

The grocery store will contain a database of items organized by aisle. Each item has a name, price, available stock quantity, and associated aisle. For example, Dairy contains milk, eggs, and cheese, while Baking contains flour, sugar, and baking soda.
The system uses these records to determine which items appear in a selected aisle and to support future stock-management features.

C. Technical Design

Proposed Items table:
• item_id — unique primary key for each item
• item_name — grocery item name
• aisle_id — foreign key referencing an aisle
• price — item price
• stock_quantity — currently available units
Each aisle has a unique aisle_id. Multiple items may share one aisle_id. This creates a one-to-many relationship between aisles and items.

D. Acceptance Criteria

1. The database contains multiple grocery aisles with corresponding items.
2. Each item has a unique ID, name, price, and stock quantity.
3. Every item belongs to a valid grocery aisle.
4. The game can retrieve items by aisle.
5. Stored item information is retrieved accurately.
6. New items can be added without altering the existing aisle structure.
GTA Testing: Inspect database records and query several aisles. Confirm that each returned item's aisle, name, price, and stock quantity match the stored information.

Artifact 2 — Requirement 8: Aisle Item Display

A. Requirement Information

Requirement ID: R8
Requirement Name: Aisle Item Display
Description: Display available grocery items when the player selects a store aisle.
Story Points: 5

B. Functional Details

When the player clicks a grocery aisle, the game displays the items assigned to that aisle. For example, selecting Dairy shows milk, eggs, and cheese. Each displayed item includes its name, price, and stock quantity. The player can select an item for quantity selection.
The system retrieves these records from the R7 database and should never display products from unrelated aisles.

C. Technical Design

Use the clicked aisle's aisle_id to retrieve matching item records from the database. Render a list or grid containing item name, price, and quantity available.
Example — Dairy Aisle:
Milk — $3.49 — 12 in stock
Eggs — $4.99 — 8 in stock
Cheese — $2.99 — 15 in stock
Provide a Back to Store control to return to the overall aisle layout.

D. Acceptance Criteria

1. Clicking an aisle opens its corresponding item display.
2. Only items belonging to the chosen aisle appear.
3. Each item shows its correct name and price.
4. Stock quantities match the database.
5. The player can select an individual item.
6. The player can return to the main store layout and choose another aisle.

GTA Testing: Open multiple aisles and check their displayed items against database records. Select an item and verify its details. Return to the store layout.

Artifact 3 — Requirement 9: Randomized Customer Order Generation

A. Requirement Information

Requirement ID: R9
Requirement Name: Randomized Customer Order Generation
Description: Generate randomized customer grocery orders containing items and required quantities.
Story Points: 8

B. Functional Details

Starting a shopping session generates a grocery order with randomly selected items and requested quantities. Every ordered product must exist in the store database. For example, an order may include two cartons of milk, one carton of eggs, three apples, and one bag of flour. The order stays fixed during the shopping session; a new session can produce a different order. Sprint 2 uses a standard itemized shopping list. Recipe-based or advanced difficulty modes are later features.

C. Technical Design

Proposed Order Items structure:
• order_id — identifier of the customer order
• item_id — requested grocery item
• required_quantity — quantity requested
• collected_quantity — quantity currently collected (alternatively derived from the cart)
Example order:
Milk — 2
Eggs — 1
Apples — 3
Flour — 1
Generate valid item IDs from the database and assign positive required quantities.

D. Acceptance Criteria

1. A new shopping session generates an order.
2. Each order includes multiple grocery items.
3. All requested items exist in the database.
4. Each required quantity is a positive integer.
5. Across repeated sessions, order combinations can vary.
6. An active order does not change unexpectedly while shopping.
GTA Testing: Start multiple shopping sessions; compare orders for variety and valid item records. Confirm that the active order remains unchanged when moving between aisles.

Artifact 4 — Requirement 10: Grocery Order Checklist

A. Requirement Information

Requirement ID: R10
Requirement Name: Grocery Order Checklist
Description: Display the active grocery list and track collection progress.

Story Points: 5

B. Functional Details

The game displays the current customer order as a checklist. Each row shows an item name, requested quantity, and collected quantity. When the player adds an item to the cart, the corresponding progress updates. An item is marked complete when the collected amount meets its requested amount. Removing items can revert its completed status.

C. Technical Design

Read order information from R9 and collected amounts from the shopping cart in R11. Compare collected_quantity against required_quantity for each item.
Example checklist:
✓ Milk — 2/2
✓ Eggs — 1/1
○ Apples — 1/3
○ Flour — 0/1
Display a completion indicator for each row and optionally a summary such as '2 of 4 items complete'.

D. Acceptance Criteria

1. The current order appears as a checklist.
2. Each requested item shows its required quantity.
3. The checklist shows collected quantities.
4. Adding an ordered item updates its progress.
5. An item is marked complete when the requested amount is collected.
6. Removing collected items updates progress and completion status.
7. Checklist progress matches cart contents.
GTA Testing: Add and remove ordered grocery items, checking each checklist quantity and completion mark after the change. Test quantities below and exactly at the required amount.

Artifact 5 — Requirement 11: Item Quantity Selection and Shopping Cart

A. Requirement Information

Requirement ID: R11
Requirement Name: Item Quantity Selection and Shopping Cart
Description: Allow players to choose item quantities and add or remove groceries from a cart.
Story Points: 5

B. Functional Details

Clicking an item in a grocery aisle opens a quantity selector. The player chooses how many units to collect and adds them to the shopping cart. The cart lists selected items and quantities, and the player can remove items or adjust quantities.
The game prevents the player from collecting more units than are available. The cart persists when the player navigates between aisles and stays synchronized with the R10 order checklist.

C. Technical Design

Proposed shopping-cart record:
• item_id — unique item identifier
• collected_quantity — current number of units in cart
Example selection interface:
Milk — $3.49 each
Available stock: 12
Quantity: [−] 2 [+]
[Add to Cart]
Check available stock before adding units; update cart records and refresh the order checklist. A quantity selection begins at one, while a cart item may be removed down to zero.

D. Acceptance Criteria

1. Clicking an item opens a quantity selector.
2. The player can increase or decrease the selection.
3. The selected add quantity cannot be below one.
4. The cart cannot exceed available stock for an item.
5. Adding items updates cart contents.
6. Removing items updates or removes the cart record.
7. Cart changes update the order checklist.
8. The cart persists when navigating between aisles.
GTA Testing: Add several item quantities from different aisles, remove units, and test the stock limit. Confirm cart persistence and checklist synchronization.

Artifact 6 — Requirement 12: Cart Validation

A. Requirement Information

Requirement ID: 12
Requirement Name: Cart Validation
Description: Validate cart against order requirements and identify missing or incorrect items.
Story Points: 5

B. Functional Details

The player can review whether the shopping cart matches the customer's active grocery order. The system compares the requested products and quantities against the cart. It identifies missing items, insufficient quantities, and additional or incorrect products so the player can correct the cart before checkout. This requirement covers validation feedback; checkout and early-exit penalties are separate future functionality.

C. Technical Design

Use the order_id to load required item IDs and quantities, and compare these against the cart's item IDs and collected quantities. Show a result for each requested item: Complete, Missing, or Insufficient Quantity. Flag cart items not in the order as Extra. Example: Milk 1/2 — Insufficient; Eggs 1/1 — Complete; Bread 0/1 — Missing; Chips 1 — Extra.

D. Acceptance Criteria

1. Validation checks every requested item against the cart.
2. A matching item and quantity are marked complete.
3. Missing products are identified.
4. Insufficient quantities are identified with the amount needed.
5. Extra or incorrect products are identified.
6. Validation results change when the cart contents change.
7. An exactly matching cart is reported as complete.

GTA Testing: Generate an order, test an empty cart, a partially filled cart, a cart with extra items, and an exactly matching cart. Verify accurate results after each change.

Artifact 7 — Requirement 16: Countdown Timer and Order Expiration

A. Requirement Information

Requirement ID: 16
Requirement Name: Countdown Timer and Order Expiration
Description: Add visible countdown timer and order expiration rules.
Story Points: 5

B. Functional Details

Each order has a limited time available for shopping and delivery. The remaining time appears on the game screen and counts down during active gameplay. When the timer reaches zero, the

order expires, a visible expiration message appears, and the game prevents the order from being completed successfully. The player's order state must remain consistent after expiration.

C. Technical Design

Store order_deadline or remaining_seconds and an order_status such as ACTIVE or EXPIRED. Initialize the countdown on order start and update the display at a consistent interval. When remaining time reaches zero, set the display to 00:00, mark the order EXPIRED, and block successful completion. Exact starting duration should be configured by the team rather than hard-coded into this artifact.

D. Acceptance Criteria

1. The timer is visible during an active order.
2. The timer starts when an order begins.
3. The remaining time decreases while the order is active.
4. The display does not show negative remaining time.
5. At zero, the system marks the order expired and displays a message.
6. An expired order cannot be completed successfully.
7. Starting a new order initializes a new countdown.

GTA Testing: Start an order and confirm the timer decreases. Use a short test duration to reach zero; verify the expiration message, status change, nonnegative display, and inability to complete the expired order.

Artifact 8 — Requirement 17: Order Earnings and Tips

A. Requirement Information

Requirement ID: 17
Requirement Name: Order Earnings and Tips
Description: Calculate order earnings and tips based on speed and order accuracy.
Story Points: 8

B. Functional Details

After an order is completed, the game calculates the amount earned based on how quickly and accurately the player fulfilled it. Faster, correctly completed orders should earn more than slow or incomplete orders under comparable conditions. The result screen displays base earnings, tip or bonus, any applicable deductions, and total payout. An expired order must not receive a successful-completion reward.

C. Technical Design

Keep base_pay, tip_amount, time_remaining_at_completion, order_accuracy, deductions, and total_earnings in the order result. Proposed calculation: total_earnings = max(0, base_pay + tip_amount - deductions). Define a configurable scoring rule for tip_amount based on completion speed and item accuracy. Do not assign fixed percentages or dollar values until the team agrees on balancing rules. Calculate rewards only once for each order and persist the result.

D. Acceptance Criteria

1. Completing an order produces an earnings result.
2. The result shows base earnings, tip or bonus, and total payout.
3. Faster completion can yield a higher payout when other conditions match.
4. Correct item quantities are rewarded relative to incorrect or missing items.
5. The total follows the configured earnings formula.
6. An expired order does not receive a successful-completion reward.
7. The same completed order is not paid twice.

GTA Testing: Complete test orders at different speeds and accuracy levels using the same base pay. Compare resulting payouts, verify arithmetic, test an expired order, and ensure repeated viewing does not duplicate earnings.

Sprint Boundaries and Dependencies

Sprint 1 covers requirements 1–4 according to the spreadsheet. Sprint 2 covers 7–12, 16, and 17. Requirements 5, 6, 13–15, and other unassigned features are not included here. The implementation details are suggested designs, not additional official requirements.

.